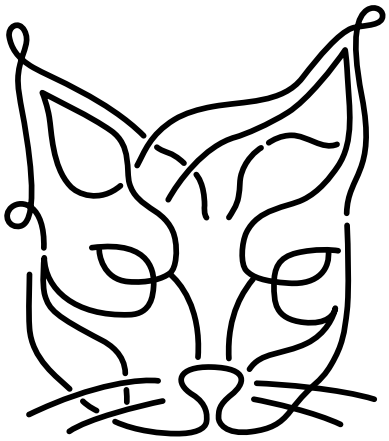

Instinto

Lince's Technical Documentation

Lince Version: 0.6.1

Typst Version: 0.14.2

Documentation Start: 2025-12-15

Documentation Print: 2026-03-28

Days of Construction: 103

Source: Lince

Contents

1. Birth ... 6
   1.1. Summary ... 7
   1.2. Installation ... 8
   1.3. Disclaimers ... 9
   1.4. Organization ... 10
   1.5. Tech Stack ... 10
2. Aging ... 12
   2.1. Record ... 13
   2.2. Collection ... 14
   2.3. Views ... 15
   2.4. Karma ... 16
   2.5. Examples ... 19
   2.6. Configuration ... 22
   2.7. Web Components ... 23
      2.7.1. Primary distinction ... 23
      2.7.2. Canonical questions for classification ... 24
      2.7.3. Runtime actors ... 24
         2.7.3.1. Backend ... 24
         2.7.3.2. Host ... 24
         2.7.3.3. Bridge ... 24
         2.7.3.4. External system ... 25
      2.7.4. Standalone widget package contract ... 25
      2.7.5. Visual and design contract ... 26
      2.7.6. Host runtime contract ... 26
         2.7.6.1. Runtime environment ... 26
         2.7.6.2. Local persistence fallback ... 27
         2.7.6.3. Bridge helper ... 27
         2.7.6.4. Datastar bridge integration ... 27
      2.7.7. Backend authentication and permissions ... 28
         2.7.7.1. Ownership ... 28
         2.7.7.2. Host-provided identifiers ... 28
         2.7.7.3. Host-recognized permissions ... 28
         2.7.7.4. Auth endpoint ... 29
      2.7.8. Backend endpoint contract ... 29
         2.7.8.1. Generic backend endpoints ... 29
         2.7.8.2. Host-mediated widget routes ... 29
         2.7.8.3. Recommended official widget runtime direction ... 30
      2.7.9. Current schema and table contract ... 30
         2.7.9.1. Generic table rules ... 30
         2.7.9.2. Current core schema reference ... 30
      2.7.10. SSE contract ... 31
         2.7.10.1. Current view SSE behavior ... 31
         2.7.10.2. Mismatch behavior ... 32

2.7.11. Recommended sidecar-table evolution ... 32
2.7.11.1. Primary rule ... 32
2.7.11.2. Recommended data layering ... 32
2.7.11.3. Minimal core table ... 32
2.7.11.4. Sparse sidecar metadata ... 32
2.7.11.5. Relational and repeated side tables ... 33
2.7.11.6. View contract strategy ... 34
2.7.12. Sand Classes ... 34
2.7.12.1. Purpose ... 34
2.7.12.2. Resource scale ... 34
2.7.12.3. Resource matrix ... 35
2.7.12.4. Five-question matrix ... 35
2.7.12.5. Engineer ... 36
2.7.12.6. Clown ... 37
2.7.12.7. Monk ... 37
2.7.12.8. Mercenary ... 38
2.7.12.9. Astromancer ... 38
2.7.12.10. Practical hybrid naming ... 39
2.7.13. Authoring rules for AI and implementation work ... 39
2.7.13.1. General ... 39
2.7.13.2. For imported standalone widgets ... 39
2.7.13.3. For official `sand/` widgets ... 39
2.7.13.4. Current widget mapping ... 39
2.7.14. Final operational summary ... 39
2.8. Kanban ... 40
2.8.1. Purpose ... 40
2.8.2. Chosen class ... 40
2.8.3. Responsibility split ... 41
2.8.4. Runtime contract ... 41
2.8.5. Data contract ... 42
2.8.5.1. Required columns ... 42
2.8.5.2. Optional columns ... 42
2.8.5.3. Current schema truth ... 42
2.8.6. Quantity mapping ... 43
2.8.7. Shell structure ... 43
2.8.8. Technology ownership ... 44
2.8.8.1. Maud ... 44
2.8.8.2. Datastar ... 44
2.8.8.3. JavaScript ... 44
2.8.9. Persistence ... 44
2.8.10. Card content model ... 45
2.8.11. CRUD model ... 45
2.8.11.1. Create ... 45
2.8.11.2. Edit ... 45
2.8.11.3. Move ... 46

2.8.11.4. Delete ... 46
2.8.12. Optional feature mapping ... 46
2.8.13. Connection and auth behavior ... 46
2.8.14. Layout and scrolling ... 47
2.8.15. UX states ... 47
2.8.16. Patch strategy ... 47
2.8.17. Functional checklist ... 48
2.8.18. Recommended implementation phases ... 48
2.8.19. Final position ... 49
2.9. Transfer Proposals ... 49
2.10. History ... 50
2.11. Dna ... 50
3. Sickness ... 51
3.1. Good practices ... 52
4. The Death of Lince ... 53
4.1. Serious Mode ... 54

1. Birth

It's a Lince!

— Docta

1.1. Summary

Lince is a tool for registry, interconnection and automation of Needs and Contributions with open scope. The phrase above is the densest way to explain Lince. The current page serves as an overview, and the subsequent chapters cover it in much more detail. Let's start with the registry part:

Registry
Users will start their Lince application for the first time, and that will create a personal database (for UNIX systems [Linux and macOS] it is located at ' /.config/lince/lince.db').

That database is a Sqlite file, that is reffered by the application as a DNA. That file can be altered to make the app behave differently. The behavior can be of: setting personal/professional tasks, modeling personal items, scheduling computer actions, act out economic trades between parties, and many more, the sky, your imagination and computing power is the limit.

Any app behavior mentioned above can be boiled down to a Need, or a Contribution to a Need. There are many ways to use the application for that end, one of them is the following: the differentiator of need or contribution is a Quantity. If such quantity is negative, that will be a Need. Having −2 Apples means you need to go and get two apples to be neutral, to satiate your needs. The same applies to a habit, having a −1 Quantity in Exercise means you need to exercise. The previous description is the basis of the whole app, everything revolves around it.

Automation
If there is predictability in your Needs, and a fixed frequency is enough to describe them, then a simple automation can be constructed. One can set the automatic every day change of the 'Studying' habit's Quantity to −1. And with another feature, if that Quantity is −1 a 'Command' is sent to the computer that closes all programs that are not related to it.

Another point of automation is the forementioned DNA. One aspect of constructing systems is that many great ideas often are lost with time, memory and the passing away of those contributors. Common patterns and knowledge about production and maintenance of commodities, economic trades, access to resources and ways of living is scattered.

These patterns could be saved into some form of database, that when fed into a program like Lince, will create Needs and Contributions, so users can have a headstart in any endeavor. From production according to a bill-of-materials to morning routines and parties organization, these activities create Needs for gathering resources and acting out tasks.

Contributions to Lince DNAs are like creating blueprints for others to get up and running. They might not be a perfect fit, but allow for great customization. Currently the way of centralizing DNAs is the Github Repository.

Interconnection
With great customizability and automation of people's DNAs, the next barrier is the connection of those individual apps, turning each machine into a p2p node, also called a

Lince Cell. The same way you create triggers and rules for the quantites of your DNA, you can do so for others.

If one Market has 200 apples and you need 5, you can create a Transfer Proposal. The Quantity of money diminishes on your side, and the Apple one increases. On the Market side, a manual price can be inputed and accepted when proposals come, or it can be adaptive. Donations of clothes, and borrowing tools for quick use would be similar exchanges, just without the money part.

1.2. Installation

You can run Lince as an executable, here are the releases. Pick the latest one for your machine and operating system. After the download, unzip it and run the binary:

```
./lince
```

Or you can install from the crates.io repository with:

```
cargo install lince
```

If you prefer to compile the source code, the easiest way is to download the repo and run:

```
cargo run
```

If you want local authentication enabled during setup, create or edit `~/.config/lince/lince.toml` and set:

```
[auth]
enabled = true
```

When auth is enabled, Lince will require an admin user to exist. If none exists yet, run `lince` once in an interactive terminal so it can prompt you to create the initial admin account.

If you don't have Cargo installed, you can install it with Rustup. It comes with Rust and its toolchain. If `cargo` isn't in your `PATH`, add it to your shell configuration (`~/.bashrc` or `~/.zshrc`) so you don't have to type the full path every time (i.e., `~/.cargo/bin/{program}`). Run this in your terminal:

```
echo 'export PATH="$HOME/.cargo/bin:$PATH"' >> ~/.bashrc
echo 'export PATH="$HOME/.cargo/bin:$PATH"' >> ~/.zshrc
```

This uses Typst to generate the documentation. One way of running the project is with Mise, the command runner that installs the programs you need to run. To get Mise, run:

```
curl https://mise.run | sh
```

Then run `mise docs` for documentation.

Lince Config Whenever you start Lince, it checks if a `lince.toml` file exists. It is in the config directory of your system, plus the lince/ directory.

Properties that may be in there are:

- `auth.enabled`: turns local authentication on or off.
- `auth.secret`: the secret used to sign the local auth session. Lince can generate it for you on first run.

Example:

```toml
[auth]
enabled = true
```

Bucket storage is not configured in `lince.toml`. It lives in the `configuration` table inside the app database, in the row with `quantity = 1`.

To configure a bucket, set these fields there:

- `bucket_enabled`: set to `1` to enable bucket storage.
- `bucket_username`: the access key / username for the bucket service.
- `bucket_password`: the secret key / password for the bucket service.
- `bucket_uri`: the bucket endpoint, such as `http://localhost:9000` for a local S3-compatible server.
- `bucket_name`: optional bucket name. If omitted, Lince uses `lince`.
- `bucket_region`: optional region. If omitted, Lince uses `us-east-1`.

If you are editing the database manually, update the active configuration row so it looks like this:

```sql
UPDATE configuration
SET bucket_enabled = 1,
    bucket_username = 'ACCESS_KEY',
    bucket_password = 'SECRET_KEY',
    bucket_uri = 'http://localhost:9000',
    bucket_name = 'lince',
    bucket_region = 'us-east-1'
WHERE quantity = 1;
```

1.3. Disclaimers

All the repositories in the Lince ecosystem have the GNU GPLv3 license. Lince is a non-profit project and crowdfunding is the source of development compensation:

GitHub Sponsors | Patreon | Apoia.se

Lince tries to facilitate and automate the connection between people and resources, by transforming needs and contributions into data. The gains and losses related to the interaction, such as transportation, production and services themselves, remain the responsibility and risk of the parties involved.

1.4. Organization

The workflow of contributing to the Lince ecosystem can be of simply making a Pull Request. But if one wants to have certainty of developing a wanted feature, the best way is to see the DNA of Lince programming tasks.

The online resource to communicate is the Discord.

The git repositories hosted currently in the GitHub organization mean this:

- lince: the app's code and this documentation.
- lince-social.github.io: the website, at lince.social.
- dna: the databases for different Lince behaviors.
- lingua: A future programming language Lince evaluates to execute karma, transactions and more.
- .github: general stuff, like photos, icons and information about GitHub sponsors.
- den: git submodules for all the repos in one place.
- rustart: A template to start a Rust project.

General Links

1.5. Tech Stack

Backend:
- Programming Language: Rust
- Database: Sqlite | Easiest to replicate the concept of DNA. Way too simple, way too reproduceable.
- Database Driver: SQLx | Used for raw and typesetted queries

Architecture:
- Application
  - Services for anything complex, business ruly, that calls repositories or manipulates data.
- Domain:
  - Entities:
    - Clean: the most accurate representation of the main drivers of the application. The source of truth of Lince's concepts.
    - Dirty: structure of data that comes in or goes out through the endpoints, or an alias for a data structure as a Type so it doesnt clutter the screen.
- Infrastructure:
  - Database:
    - Management: database connection, migrations and schema.
    - Repositories: traits and their implementations for database operations like get, update…
  - Utils: functions for quality of life, like logging.

  - Cross_cutting: dependency injection for a collection of many methods in different layers. Mostly for single database connection initiation.
- Presentation
  - GPUI: GPU accelerated Rust dependency used by Zed.

(Some Rust) Learning Resources

1. Rust Book: chapters 1–6 will give you a simple base if you just want to hack something together. Passing through the entire book, doing the examples and understanding them deeply, will set you up for great vibe coding.
2. Rustlings: for when you are done with the book. You will complete exercises on a broad spectrum of the language's features.
3. Rust for Rustaceans: for a deeper dive in the language, good practices and crate recomendations.
4. This Week in Rust: a weekly newsletter about Rust projects, crates and language changes. Amazing for keeping up to date with the ecosystem, very useful.
5. Youtube Channels: Jon Gjengset | No Boilerplate | The Rustagen | Tsoding | Code to the Moon | fasterthanlime | chris biscardi | Mike Code | Developer Voices
6. Podcast: Rustacean Station
7. Discord, Matrix and many more online communities are very welcoming to newbies.

2. Aging

Ouch! My back!

— Sedentary Person, Modern

This chapter explains Lince's components in more detail. It is the worse chapter by far. The project is too much in an Alpha stage and the Schema of the Database changes too much and a definitive one hasn't been chosen (was Postgres at first, now it's SQLite). Some of the properties in here might be out of date or not implemented yet. Explanation of the tables *should* be mostly focused on the ideas, the solutions brought to a problem yet to be found.

2.1. Record

record
id
quantity
head
body
location
time

Lince is centered on the 'record' table, but like, according to the creator… like… what does he know?

A 'record' can be anything: an action, an item, a plan.

record	DATA TYPE
id	INTEGER
quantity	FLOAT
head	TEXT
body	TEXT
location	3D POINT (still thinking)

'id's are automatically generated.

'quantity' represents the availability of the record, if negative it is a Necessity, if positive, a Contribution, zero makes it.

'head' and 'body' are meant to be parts of a whole, where one can be used for a short summary and the other a description, or one has all the information and the other holds tags for filtering through views. With a pubsub protocol, one can send a short information of the record, in this case it can be the head, and put the rest in the body. Only those interested in the head will ask for the body of the record. That way the minimum amount of information is sent over the network, making it faster and stuff, I think.

'location' is an important information for interactions outside of computers (they exist, it's insane) or any other use you want to give it. 'time' is a property like location, not implemented yet. It could be a hard-coded property that maps to a concept in the real world. Not a generic one like quantity. It's probably better to just have multiple custom properties attatched to a record, like cost of time, location, cost of money as data on the

user's DNA, not as code in Lince repo. If we do it in a generic way, each record can have many different properties Lince doesn't have to predict to fit many use cases. The question is, how to we interlink a record with those properties? Other records? Such properties need to have the effect of their names: negative Time in Record A needs to remove that cost of time from the user's Time Record's quantity?

record	DATA TYPE	USER INPUT
id	INTEGER	
quantity	FLOAT	−1
head	TEXT	Eat Apple
body	TEXT	
location	POINT	

So, for an example, imagine that you like apples and you want to create a task to eat it today.

You create a 'record', giving it '−1' to the 'quantity', for that action is a Necessity in your life right now, and 'Eat Apple' to the 'head'.

—

record	DATA TYPE	USER INPUT	ACTUAL RECORD
id	INTEGER		1
quantity	FLOAT	−1	−1
head	TEXT	Eat Apple	Eat Apple
body	TEXT		NULL
location	POINT		NULL

The end result, on the database, is this record.

—

Here is an example of different possible records for individual items and actions.

id	quantity	head	body	location
1	−1	Eat Apple	NULL	NULL
2	−1	Apple	Item, Food	NULL
3	0	Brush Teeth	Action, Hygiene	NULL
4	3	Toothbrush	Item, Hygiene	NULL
5	−1	Meditate	Action	NULL

2.2. Collection

COLUMNS	DATA TYPE

id	INTEGER
name	TEXT
quantity	INTEGER

A Collection is the name of a list of Views. Only one Collection is active at a time, with quantity 1, the rest is 0.

2.3. Views

COLUMNS	DATA TYPE
id	INTEGER
name	TEXT
query	TEXT

Views are ways to select data, to see what records exist with filters or ordering… They can also be used to see what Collections, Karma, Configurations… exist.

A view will have a name, so it is human readable and a query to display different data.

The query is made with SQL allowing you to select what columns you want to see, filtered, ordered and much more, just the way you want it.

The 'query' of a view can also be a special code, like 'karma_view', in the HTML (Web Browser) frontend. It is a specially constructed html to interact with Karma in a more complete and sane way, to bring ergonomics to day-to-day Lince. Instead of returning an automatically constructed table with data from an SQL query, the Backend adds customized HTML components to the page sent to the user (normal frontend behavior).

Examples:

Let's grab the record table shown before.

id	quantity	head	body	location
1	−1	Eat Apple	NULL	NULL
2	−1	Apple	Item, Food	NULL
3	0	Brush Teeth	Action, Hygiene	NULL
4	3	Toothbrush	Item, Hygiene	NULL
5	−1	Meditate	Action	NULL

A view of things you need to buy may look like this:

COLUMN	VALUE
id	1
name	Buy

query	SELECT * FROM record WHERE LOWER(body) LIKE ‘%item%’ AND quantity < 0

And in this case, will display only:

id	quantity	head	body	location
2	−1	Apple	Item, Food	NULL

Since no other records with a body containing ‘Item’ (lower, upper, pascal case…) exist with a negative quantity.

Collection View

COLUMNS	DATA TYPE
id	INTEGER
quantity	INTEGER
collection_id	INTEGER
view_id	INTEGER

A Collection View is an intermediate table for grouping Views into a Collection. The quantity is the order in which they appear and if positive they are shown, negative ones are not.

2.4. Karma

karma	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
condition_id	INTEGER
operator	TEXT
consequence_id	INTEGER

Karma is a condition checker and an action taker. A constructor of if/then, behavior, inside your DNA. It does it by selecting data or running commands, then if the data fits some criteria, changing records, running commands and more. It replaces a syntax that references parts of Lince with it’s actual values, then evaluating it as a mathematical equation and according to the Operator. If the operator is ‘=’ it lets only non zero values take effect, if it is ‘=*’ it doesnt have that constraint.

This is the hardest thing to understand in Lince and also the most useful. The full description, plus examples, will take place a little further ahead. This process is called a Delivery and it is run every 60 seconds, and sometimes instantly when wanted.

Karma Condition

COLUMN NAME	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
condition	TEXT

A Karma Condition is something checked by replacing parts of the string (text) with real values. A record with id 1 has a quantity of 5. When we set a Karma Condition to be 'rq1' we are saying the value evaluated will be 5 (at that Delivery).

Karma Consequence

COLUMN NAME	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
consequence	TEXT

The Karma Consequence works the same way as Condition but instead of getting values it is responsible for setting what is supposed to change. It can be the activation of a Shell/SQL command, the changing of the value of a Record and more; in the future it would be cool to make transactions, changing Quantities of other tables like Configuration and Collection Views, maybe even altering other Karma, or changing DNAs.

Frequency

frequency	DATA TYPE
id	INTEGER
quantity	INTEGER
day_week	INTEGER
months	INTEGER
days	INTEGER
seconds	INTEGER
next_date	STRING
finish_date	STRING
catch_up_sum	INTEGER

The frequency table holds account of a fixed period for returning a value of 1 or more. If a frequency occurs every 1 day and 60 seconds it might start, for example at 2030-01-01 10:00:00. When that time comes, if this Frequency exists in a Karma Condition it will be checked and updated, setting next_date to 2030-01-02 10:01:00.

The catch_up_sum is a multiplier, if many days have passed since the last check like 4, instead of returning one each minute, it catches up to the present in 'next_date' and returns the periods passed or only one.

In Karma, Frequencies are represented by the letter ‘f’. So setting in a Condition ‘−1 * f1’ means Karma Delivery will check the returned number for the specific Frequency with Id 1 and multiply that by ‘−1’.

Command

command	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
command	TEXT

The Command is a Shell command you can run in a bash Shell.

Example:

id	quantity	command
1		touch grass.el

It is referenced in Karma Condition and/or Consequence as the letter ‘c’, followed by the id number, so this example would be ‘c1’. The command above creates the file grass.el.

Query

COLUMN NAME	DATA TYPE
id	INTEGER
quantity	INTEGER
query	TEXT

The Query is an SQL command you can run and affect your current DNA.

Example:

[“id”, “quantity”, “query”], [“1”, “”, “Robert’); DROP TABLE users; −”],

It is referenced in Karma Condition and/or Consequence as the characters ‘sql’, followed by the id number, so this example would be ‘sql1’.

Sum

sum	DATA TYPE
id	INTEGER
quantity	INTEGER
record_id	INTEGER
interval_relative	BOOL
interval_length	INTERVAL
sum_mode	INTEGER

end_lag	INTERVAL
end_date	TIMESTAMP

This table is responsible for returning a sum of change. Change of a record's quantity over time. You have the 'record_id' to know what record to look for. 'interval_length' is a period, can be 1 day, 8 months, etc. If 'interval_relative' is TRUE, will sum all the changes ending now, starting at the past based on the 'interval_length' (ex: 'interval_length' of 8 months and 'interval_relative' is TRUE will make it look at the past 8 months). If FALSE an 'end_date' will need to be supplied. so the past 8 months ending in a certain date. 'end_lag' will give a space between now and the end point of a relative interval. So if the 'interval_length' is 8 months and 'interval_relative' is TRUE, an 'end_lag' of 2 months will make the sum look at the 2 months back to 10 months back, starting 10 months ago, ending 2 months ago. 'sum_mode' tells Lince what to sum, if 0 it's a delta, essentially doing a quantity now - quantity earlier. If it's negative, it sums all negative changes, positive, idem.

2.5. Examples

So you now know the parts of Karma. It's time to put everything together. Lets take this example:

Record

id	quantity	head	body
1	0	Exercise	

Frequency

id	quantity	days	next_date
1		1	2030-01-01 10:00:00

Condition

id	quantity	name	condition
1		Daily	−1 * f1

Consequence

id	quantity	name	consequence
1		Exercise (Automatically set)	rq1

Karma

id	quantity	name	condition_id	operator	consequence_id
1		Daily Exercise (Automatically set)	1	=	1

With this data, every Karma Delivery (60 seconds) will check the validity of each condition. Frequency with id 1 ‘f1’ will return 0 in all minutes of the day safe for one time, where it will return 1. Since the operator of the Karma with id 1 is ‘=’ the expression in it’s Condition ‘−1 * f1’ will almost always be ‘−1 * 0’ and will not have a consequence.

In the Karma delivery that is past the next_date of Frequency, the next_date is updated and the Condition is ‘−1 * 1’. Since that is equal to ‘−1’ the Consequence ‘rq1’ will change the quantity of Record with Id 1, turning it into ‘−1’.

We go from this:

id	quantity	head	body
1	0	Exercise	

To this:

id	quantity	head	body
1	−1	Exercise	

One could make Conditions like ‘(rq1 - 1) * f1’ and a Consequence of ‘rq2’ so the number would not be fixated at −1, but would decrease by one every day based on rq1. The expressions can become very complicated, but unless you are doing finance, joinning multiple sources of data into two or three records like monthly costs/gains and runaway you will do simple expressions.

Example: Command

With the command table

Let’s say you have a command:

id	quantity	name	command
3	1	Radarada	touch grass.el

This command with ID 3 can be referenced in a Karma Consequence with ‘c3’.

Consequence

id	quantity	name	consequence
2		Radarada (Automatically set)	c3

We can reuse the Frequency, and Condition, creating just a Consequence and Karma to automate the running of this command.

Frequency

id	quantity	days	next_date
1		1	2030-01-01 10:00:00

Condition

id	quantity	name	condition
1		Daily	−1 * f1

Karma

id	quantity	name	condition_id	operator	consequence_id
1		Daily Radarada (Automatically set)	1	=	2

This will daily run the Command ‘touch grass.el’.

—

Alternativelly, you can make it so the Command is part of the Condition.

```
echo $(find ~/books/technology -type f | wc -l)
```

This command will output a number, which we can use inside a Condition.

Command

id	quantity	name	command
4		Tech Books Count	echo $(find /books/technology -type f | wc -l)
5		Open Current Tech Book	pdfreader /books/techology/current/*

Condition

id	quantity	name	condition
3		Read Tech Task	−1 * f1 * c4

Consequence

id	quantity	name	consequence
3		Open Current Tech Book	c5

Karma

id	quantity	name	condition_id	operator	consequence_id
1		Daily Radarada (Automatically set)	3	=	3

That will automatically open the current technology book one is reading. If the automatic opening is not ideal one can set it so the Consequence is changing the quantity of a Record:

id	quantity	head	body
1	−1	Read Tech Book	

When typing 1 in operation the quantity of this Record will be zero. If you have a Condition that is ‘rq1 == 0’ that will be evaluated to ‘1’ and can trigger the Consequence

‘c5’. Though this method needs one Karma expression to run ‘c5’ if ‘rq1 == 0’ and another to make it so if ‘rq1 == 0’ Condition then ‘rq1’ Consequence, making ‘rq1’ be ‘1’ and not triggering the infinite opening of the Tech Book. In simpler terms: you set two different Karma, one checks if ‘Read Book’ has quantity 0, if yes, make it 1. The other Karma opens the book if the quantity of ‘Read Book’ is zero. So when we set it through zero it will open the book once, for we have a rule that quickly moves it to not zero.

2.6. Configuration

configuration	DATA TYPE
id	INTEGER
quantity	INTEGER
language	TEXT
timezone	TEXT
style	TEXT

The Configuration table alters some behavior of the Lince application. Language and Timezone are what they appear, and style is for setting what colorscheme is used.
Operations

There is a keyboard way of navigating through Lince, changing records and running commands. That is done through the Operations. When one types anything in the frontend, an input will appear.

If you type ‘4c’, a modal for creating a Record will pop up. Typing ‘a’ with a number, like ‘a2’ will make Configuration with id 2 be active, and others inactive, an easy way to change colorscheme. One handy one is ‘s’ with a number, ‘s1’ to run a Shell command.

Just typing a number, like ‘23’ will make the Record with id 23 have it’s quantity become zero.

Writting ‘k3’ will change the active collection to be the one with Id 3, effectivelly changing screens.

Its possible to click in the collection to change it, and setting the quantity to zero by manually editing the field in the table. But those operations are very frequent and can be more ergonomic for those that like to remember shortcuts. Those that dont will have a hard time currently :(

[#] Name	[Key] Action
[0] Configuration	[c] Create
[1] Collection	[q] SQL Query
[2] View	[k] Change Collection
[3] collection_View	[s] Shell Command
[4] Record	[a] Activate Configuration

[5] Karma_Condition	
[6] Karma_Consequence	
[7] Karma	
[8] Command	
[9] Frequency	
[10] Sum	
[11] History	
[12] DNA	
[13] Transfer	

2.7. Web Components

This document is the canonical technical specification for creating web components in Lince.

Use it as AI-facing reference material.

Current code is more authoritative than this document when there is conflict. This document exists to provide one dense, unified view of:

- standalone widget packaging
- official host runtime behavior
- bridge and state contracts
- backend authentication and endpoint usage
- current schema and SSE behavior
- recommended sidecar-table evolution
- Sand Class taxonomy
- authoring defaults for widgets that interact with Lince and foreign resources

2.7.1. Primary distinction

There are two broad families of web components:

- standalone imported widgets
- official `sand/` widgets

Standalone imported widgets are single HTML documents executed inside an iframe through `srcdoc`.

Official `sand/` widgets are Lince-maintained components, normally rendered with Rust + Maud and increasingly expected to use an internal runtime contract rather than each widget hand-rolling transport logic.

2.7.2. Canonical questions for classification

Every widget should be understood by five questions:

1. Where does truth normally live?
2. How does truth normally reach the widget?
3. Who normally shapes the UI?
4. Where does interaction state normally live?
5. How portable or foreign is the runtime?

Do not invent a new Sand Class because a widget has a different domain or a different visual style. Only create a new class if at least two of those five answers fundamentally change.

2.7.3. Runtime actors

2.7.3.1. Backend

The backend owns domain truth.

It normally owns:

- persisted business data
- generic table CRUD
- saved views
- view streaming
- semantic actions when they exist

2.7.3.2. Host

The host is the Lince web runtime that embeds the widget.

It normally owns:

- widget instance identity
- permissions
- auth state
- board layout
- persisted `widgetState`
- stream enable or pause state
- locked/configure/misconfigured host states

2.7.3.3. Bridge

The bridge is the host-to-widget communication layer.

Treat it as control plane, not bulk data plane.

Use it for:

- bootstrap state
- runtime metadata
- auth and permissions
- `widgetState` persistence
- stream enable or pause requests
- subscriptions to host state updates

Do not use it as the main large-record snapshot transport when the backend or official stream is the correct source.

2.7.3.4. External system

Anything outside Lince, such as:

- third-party APIs
- foreign websites
- imported HTML
- mounted foreign apps
- local files
- browser APIs
- WebAssembly runtimes

2.7.4. Standalone widget package contract

When generating standalone imported widgets, the package contract is:

- one complete HTML document
- self-contained
- safe to save directly as a `.html` file
- executed inside an iframe via `srcdoc`

Hard requirements:

- do not depend on remote CDNs
- do not depend on remote fonts
- do not assume network access unless the prompt or host contract explicitly provides it
- do not render a second outer browser frame or a fake card inside the card
- use inline CSS and inline JS when generating a standalone HTML widget
- do not call `parent` window APIs directly
- use the injected bridge if host communication is needed

The embedded manifest block must exist near the top of the document:

```
<script type="application/json" id="lince-manifest">
  {
    "title": "Widget title",
    "author": "Author",
    "version": "0.1.0",
    "description": "Short summary",
```

```
    "details": "Longer explanation",
    "initial_width": 4,
    "initial_height": 3,
    "permissions": []
  }
</script>
```

Manifest fields:

- `title`: short user-facing name
- `author`: short author string
- `version`: semantic version string
- `description`: compact summary
- `details`: longer explanation
- `initial_width`: integer between `1` and `6`
- `initial_height`: integer between `1` and `6`
- `permissions`: host-recognized capabilities

2.7.5. Visual and design contract

The host visual language for imported or official widgets is:

- dark
- minimal
- technical
- compact
- premium rather than playful by default

Preferred visual behavior:

- use restrained color
- avoid gradients unless explicitly wanted
- make small card sizes viable
- do not overflow aggressively
- remain useful in preview mode
- use strong information hierarchy
- prefer compact controls

2.7.6. Host runtime contract

2.7.6.1. Runtime environment

When a widget runs inside the board:

- the widget runs in an iframe through `srcdoc`
- the host injects a vendored Datastar runtime
- the host injects `window.LinceWidgetHost`
- the widget may receive dataset metadata on the frame element

Do not add a CDN Datastar import.

2.7.6.2. Local persistence fallback

If internal widget state must use `localStorage`, namespace it with the widget instance:

```
const instanceId = window.frameElement?.dataset?.packageInstanceId ||
"preview";
```

Prefer host-persisted state through the bridge instead of relying only on `localStorage`.

2.7.6.3. Bridge helper

The host bridge helper lives at:

`window.LinceWidgetHost`

Bridge surface:

- `getState()`
- `getMeta()`
- `getCardState()`
- `setCardState(nextState)`
- `patchCardState(patch)`
- `setStreamsEnabled(enabled)`
- `subscribe(handler)`
- `requestState()`
- `print(label)`

Bridge persistence rules:

- prefer `patchCardState()` or `setCardState()` for per-widget runtime state
- keep long-lived visual ergonomics in `widgetState`
- do not store backend credentials or backend configuration copies there

2.7.6.4. Datastar bridge integration

If reactive host state is needed:

- add `data-lince-bridge-root` to a root element
- listen for `lince-bridge-state`

Event shape:

`{ bridge, meta }`

Datastar example:

```
<div
  data-lince-bridge-root
  data-on:lince-bridge-state="$bridge = evt.detail.bridge; $meta =
```

```
evt.detail.meta"
></div>
```

For Datastar-driven UI state:

- keep UI state in signals
- persist relevant state through signal-patch handlers into host `widgetState`

2.7.7. Backend authentication and permissions

2.7.7.1. Ownership

Backend authentication is host-owned.

Widgets must not:

- render their own login UI for the standard Lince backend flow
- store backend tokens in `localStorage`
- store backend tokens in widget archives
- store backend tokens in cookies
- store backend tokens in board state

Treat remote backend credentials as host-managed and in-memory only.

2.7.7.2. Host-provided identifiers

When a widget needs server-backed data, assume the host card configuration provides identifiers such as:

- `server_id`
- `view_id`

For host-configured widgets, those values may appear at:

- `window.frameElement?.dataset?.linceServerId`
- `window.frameElement?.dataset?.linceViewId`

Once the bridge is available, treat host metadata as authoritative:

- `meta.serverId`
- `meta.viewId`
- `meta.streams.enabled`

Do not create a second widget-local configuration store for server or view identity.

2.7.7.3. Host-recognized permissions

Current important permissions for backend-aware widgets:

- `read_view_stream`
- `write_records`

- `write_table`

Meaning:

- `read_view_stream`: widget consumes a configured SSE view stream and therefore needs `server_id` and `view_id`
- `write_records`: widget mutates record rows through host-mediated routes
- `write_table`: widget mutates generic tables through host-mediated routes and is the important generic permission for sidecar-table evolution

Do not invent near-miss permission names.

2.7.7.4. Auth endpoint

Current backend login endpoint:

```
POST /api/backend/auth/login
```

Request body:

```
{ "username": "...", "password": "..." }
```

Success response:

```
{ "token": "...", "token_type": "Bearer" }
```

Authenticated backend calls use:

```
Authorization: Bearer <token>
```

Widgets should not perform this login flow themselves for the normal host-mediated backend workflow.

2.7.8. Backend endpoint contract

2.7.8.1. Generic backend endpoints

Current direct backend endpoints:

```
GET    /api/backend/table/{table}
POST   /api/backend/table/{table}
GET    /api/backend/table/{table}/{id}
PATCH  /api/backend/table/{table}/{id}
DELETE /api/backend/table/{table}/{id}
GET    /api/backend/sse/view/{view_id}
```

2.7.8.2. Host-mediated widget routes

Widgets that are configured against a remote server should prefer host-mediated routes:

```
GET    /host/integrations/servers/{server_id}/views/{view_id}/stream
GET    /host/integrations/servers/{server_id}/table/{table}
POST   /host/integrations/servers/{server_id}/table/{table}
```

```
GET    /host/integrations/servers/{server_id}/table/{table}/{id}
PATCH  /host/integrations/servers/{server_id}/table/{table}/{id}
DELETE /host/integrations/servers/{server_id}/table/{table}/{id}
```

Rules:

- do not call `/api/backend/...` directly from a widget when the widget is supposed to use a configured remote server
- do not pass `server_id` as a query parameter to `/api/backend/...`
- use one obvious data source per widget unless there is a deliberate reason to mix them

2.7.8.3. Recommended official widget runtime direction

For official `sand/` widgets, the preferred long-term direction is an instance-aware runtime:

```
GET  /host/widgets/{instance_id}/stream
GET  /host/widgets/{instance_id}/contract
POST /host/widgets/{instance_id}/actions/{action}
```

Purpose:

- resolve widget config by instance
- centralize host/runtime state lookup
- consume backend streams internally
- emit Datastar fragments or signals without each widget rebuilding transport
- support richer, semantic actions instead of requiring client-side multi-table orchestration

This is recommended direction, not a claim that every route already exists in the current implementation.

2.7.9. Current schema and table contract

2.7.9.1. Generic table rules

- table endpoints are generic
- table names are path parameters
- rows are JSON objects
- mutations use JSON request bodies
- row identity is expected to use integer `id`

CRUD widgets should state which table they operate on.

If the user asks for a specific table, constrain the widget to that table.

2.7.9.2. Current core schema reference

Current important table reference derived from migrations and present docs:

- `record`: `id`, `quantity`, `head`, `body`
- `view`: `id`, `name`, `query`

- `collection`: `id`, `quantity`, `name`
- `configuration`: `id`, `quantity`, `name`, `language`, `timezone`, `style`, plus later config columns
- `collection_view`: `id`, `quantity`, `collection_id`, `view_id`, `column_sizes`
- `karma_condition`: `id`, `quantity`, `name`, `condition`
- `karma_consequence`: `id`, `quantity`, `name`, `consequence`
- `karma`: `id`, `quantity`, `name`, `condition_id`, `operator`, `consequence_id`
- `frequency`: `id`, `quantity`, `name`, `day_week`, `months`, `days`, `seconds`, `next_date`, `finish_date`, `catch_up_sum`
- `command`: `id`, `quantity`, `name`, `command`
- `transfer`: `id`, `quantity`
- `sum`: `id`, `quantity`, `record_id`, `interval_relative`, `interval_length`, `sum_mode`, `end_lag`, `end_date`
- `history`: `id`, `record_id`, `change_time`, `old_quantity`, `new_quantity`
- `dna`: `id`, `name`, `origin`, `quantity`
- `query`: `id`, `name`, `query`
- `pinned_view`: `id`, `view_id`, `position_x`, `position_y`, `z_index`, `width`, `height`

Important present truth:

- `record` is still the core generic table
- its core schema is still only `id`, `quantity`, `head`, `body`

2.7.10. SSE contract

2.7.10.1. Current view SSE behavior

Underlying backend route:

```
GET /api/backend/sse/view/{view_id}
```

Typical host-mediated route:

```
GET /host/integrations/servers/{server_id}/views/{view_id}/stream
```

Current event model:

- named SSE events
- `snapshot` carries the latest JSON payload for the view snapshot
- `error` carries a JSON payload describing a backend or subscription error

Widget rules for current SSE:

- reconnect cautiously
- keep the UI usable while waiting for first snapshot
- do not assume workspace switching remounts the iframe
- persist UI state explicitly
- validate expected schemas
- show compact mismatch state if payload shape is wrong

2.7.10.2. Mismatch behavior

When a widget expects a specific shape:

- validate it
- show compact mismatch UI
- log expected shape and received shape briefly in the console

2.7.11. Recommended sidecar-table evolution

2.7.11.1. Primary rule

Keep `record` generic.

Do not keep adding task-specific columns to `record` unless they become universally central to Lince itself.

Do not treat `record.body` as canonical structured storage for every evolving feature.

Body front matter is acceptable only as:

- import path
- migration bridge
- prototype shortcut

2.7.11.2. Recommended data layering

Use three layers:

1. minimal core table
2. sparse sidecar metadata
3. dedicated tables for repeated or relational data

2.7.11.3. Minimal core table

Keep:

- `record.id`
- `record.quantity`
- `record.head`
- `record.body`

2.7.11.4. Sparse sidecar metadata

Recommended table:

```
CREATE TABLE record_extension (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
```

```
    namespace TEXT NOT NULL,
    version INTEGER NOT NULL DEFAULT 1,
    data_json TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, namespace),
    CHECK (json_valid(data_json))
);
```

Use it for sparse, optional, versioned scalar metadata.

Recommended namespaces:

- `task.bucket`
- `task.schedule`
- `task.effort`
- `task.assignment`

Do not store component UI state there.

2.7.11.5. Relational and repeated side tables

Recommended generic relation table:

```
CREATE TABLE record_link (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    link_type TEXT NOT NULL,
    target_table TEXT NOT NULL,
    target_id INTEGER NOT NULL,
    position REAL,
    data_json TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, link_type, target_table, target_id),
    CHECK (data_json IS NULL OR json_valid(data_json))
);
```

Recommended comment table:

```
CREATE TABLE record_comment (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    author_user_id INTEGER,
    body TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    deleted_at DATETIME
);
```

Recommended worklog table:

```
CREATE TABLE record_worklog (
    id INTEGER PRIMARY KEY,
```

```
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    author_user_id INTEGER,
    started_at DATETIME NOT NULL,
    ended_at DATETIME,
    seconds REAL,
    note TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

2.7.11.6. View contract strategy

Widgets should consume normalized views, not raw storage assumptions.

Examples:

- simple record table: use only `record`
- lightweight Kanban: use `record` plus optional bucket metadata
- planning Kanban: use `record` plus sidecars and aggregates
- worklog dashboard: use `record_worklog` aggregates and optional effort data

This keeps the base schema generic while letting widgets stay strict about what they need.

2.7.12. Sand Classes

2.7.12.1. Purpose

Sand Classes are not permission boundaries. They are a compressed vocabulary for the usual data, runtime, and rendering philosophy of a widget.

Primary classes:

- `Engineer`
- `Clown`
- `Monk`
- `Mercenary`
- `Astromancer`

Hybrid naming is encouraged when it is more truthful than forcing one bucket.

2.7.12.2. Resource scale

All scale values are:

- `none`
- `low`
- `mid`
- `high`

2.7.12.3. Resource matrix

Resource	Engineer	Clown	Monk	Mercenary	Astromancer
Maud templating	high	low	none	none	none
Datastar signals in frontend	low	high	low	none	none
Datastar signals from backend	low	high	none	none	none
Datastar HTML fragments from backend	high	low	none	none	none
Raw JSON or hand-parsed data	none	none	high	low	low
Host or bridge runtime state	high	high	low	low	low
Official Lince endpoints	high	mid	low	none	low
External APIs, pages, or apps	none	low	low	high	none
Files or local resources	none	low	low	mid	mid
Iframe or embed isolation	none	none	none	mid	none
Foreign framework runtime	none	none	mid	mid	none
WASM or browser-engine runtime	none	none	none	none	high

2.7.12.4. Five-question matrix

Question	Engineer	Clown	Monk	Mercenary	Astromancer
Where does truth normally live?	Backend	Mixed: backend for business, frontend for presentation	Varied, often client-owned or remote-contract-owned	Foreign system or local widget space, with Lince optional	Resource or engine dependent
How does truth normally reach the widget?	Official backend stream and endpoints through the host	Backend seed, optional backend stream, host runtime state, and heavy signal-driven local state	Raw JSON SSE, fetch JSON, manual parsing, or vendored runtime adapters	Foreign page, foreign API, files, embeds, or adapter layers	File, buffer, resource binding, engine feed, or runtime adapter

Who normally shapes the UI?	Maud plus backend fragments and a few signals	Small shell plus frontend Datastar signals	JS or vendored framework	Foreign UI, foreign framework, or adapter runtime	JS or WASM engine, with DOM shell secondary
Where does interaction state normally live?	Frontend for ergonomics, backend for domain mutations	Mostly frontend	Varied, usually frontend	Mostly frontend or foreign runtime	Engine-local or use-case dependent
How portable or foreign is the runtime?	Low, strongly Lince-coupled	Low to mid, still fairly Lince-shaped	Mid to high, usually self-contained	High, intentionally foreign-capable	Mid, more constrained by runtime capabilities than by Lince

2.7.12.5. Engineer

Definition:

- default official class for robust Lince-native workflows

Transport and shaping:

- backend truth
- official endpoints
- Maud-shaped structure
- Datastar HTML fragments as main structural patch mechanism
- backend signals only as secondary support

JavaScript profile:

- use JS sparingly
- preferred uses: drag/drop edges, resize edges, focus-sensitive editing, browser APIs awkward in Datastar

Best fits:

- record tables
- CRUD panels
- record browsers
- search results
- audit logs
- server-owned dashboards

2.7.12.6. Clown

Definition:

- signal-heavy, frontend-reactive class

Transport and shaping:

- often mixed truth
- stable shell
- frontend Datastar signals do most of the visible orchestration
- small backend fragment use is acceptable when targeted structural patches help

JavaScript profile:

- low
- use JS for imperative browser edges around signal-driven state

Best fits:

- clocks
- counters
- compact status widgets
- dashboards with stable shells and lively presentation state
- inspectors with many local display modes

2.7.12.7. Monk

Definition:

- client-shaped class built from raw data contracts

Transport and shaping:

- raw JSON SSE or fetched JSON
- JS or vendored framework shapes the UI
- Datastar is optional and minor

JavaScript profile:

- high
- JS is the main rendering tool
- use for snapshot parsing, DOM construction, library lifecycles, client-owned interaction flows

Best fits:

- experimental widgets
- legacy ports
- client-heavy graphs
- widgets using libraries that expect raw data

2.7.12.8. Mercenary

Definition:

- integration-first, foreign-capable class

Transport and shaping:

- truth often lives outside Lince
- wraps, embeds, fetches from, or adapts foreign systems
- Lince participates opportunistically rather than exclusively

JavaScript profile:

- low to mid
- use for adapters, wrappers, embed coordination, framework mounting, and API mediation

Best fits:

- embeds
- foreign sites or apps
- wrapper widgets
- framework-backed integrations
- imported HTML with fallback behavior

2.7.12.9. Astromancer

Definition:

- runtime-heavy class for engine-driven browser surfaces

Transport and shaping:

- engine or runtime shapes the meaningful surface
- DOM shell is mostly chrome and fallback structure

JavaScript profile:

- mid
- use JS for engine integration, lifecycle orchestration, browser capability access, and glue around WASM or other runtime cores

Best fits:

- terminal widgets
- canvas surfaces
- media tools
- graph engines
- WASM-powered inspectors

2.7.12.10. Practical hybrid naming

Preferred examples:

- official Kanban: `Engineer with Clown traits`
- portable integration: `Mercenary with Monk fallback`
- terminal: `Astromancer with Engineer chrome`

2.7.13. Authoring rules for AI and implementation work

2.7.13.1. General

- prefer one obvious primary data source
- do not invent endpoints that are not present in code or explicitly requested
- do not invent columns that are not present or explicitly specified
- if schema is ambiguous, stay generic and say what is assumed

2.7.13.2. For imported standalone widgets

- self-contained HTML
- no remote dependencies
- no custom login UI for standard host-backed flows
- no parent-window APIs

2.7.13.3. For official `sand/` widgets

- default to `Engineer`
- add `Clown` traits when local presentation state becomes rich
- use `Monk` deliberately, not lazily
- use `Mercenary` when outside systems are a real product requirement
- use `Astromancer` when the browser runtime itself is the main engine

2.7.13.4. Current widget mapping

Recommended current mapping:

- `extra_simple`: `Mercenary` during migration, then `Engineer`
- `view_table_editor`: `Engineer`
- `kanban_record_view`: `Engineer with Clown traits`
- `ops_clock`: `Clown`
- `local_terminal`: `Astromancer`

2.7.14. Final operational summary

For official long-lived widgets:

- backend truth
- host-owned auth and runtime state
- bridge as control plane
- Maud for structure
- Datastar fragments for server-owned repeated structure
- Datastar signals for stable-shell runtime and local UI state
- JS only where imperative browser behavior is the right tool

For foreign-facing widgets:

- treat portability as a first-class architectural property
- do not fake an `Engineer` when the widget is really `Mercenary` or `Monk`

For engine-driven widgets:

- acknowledge `Astromancer` directly instead of forcing canvas, terminal, or WASM runtimes into ordinary DOM-first assumptions

2.8. Kanban

This document is the Kanban-specific creation and implementation specification.

It is intentionally separate from the general web component specification. Read `web_components.typ` first for:

- host/runtime assumptions
- endpoint contracts
- Sand Classes
- sidecar-table strategy

2.8.1. Purpose

This specification defines the official Record Kanban for the web version.

The target is:

- Record-centric
- fed by a view SSE source
- persisted per widget through host `widgetState`
- authenticated by the host-owned web login
- implemented as `Engineer with Clown traits`

2.8.2. Chosen class

The official Kanban should be:

- `Engineer with Clown traits`

Reason:

- server-owned board structure matters
- per-card UI state matters
- drag and drop matters
- resize matters
- connection/auth clarity matters
- local display ergonomics matter

Pure `Engineer` is too rigid for the amount of local interaction state. Pure `Clown` pushes too much structure into the browser. `Monk` makes JS the default rendering model. `Mercenary` is not the center of gravity for a Lince-native Kanban. `Astromancer` is the wrong class for a DOM-first board.

2.8.3. Responsibility split

Normal center of gravity:

- Maud: main structural owner
- Datastar: local UI state, status, selected stable-shell reactivity
- JavaScript: drag/drop, resize, focus-sensitive editing, and scroll behavior that needs imperative handling

Normal transport split:

- HTML fragments from backend or official widget stream: primary
- signal patches: secondary
- raw JSON SSE: only as narrow fallback during transition

2.8.4. Runtime contract

Preferred runtime direction:

```
GET  /host/widgets/{instance_id}/stream
GET  /host/widgets/{instance_id}/contract
POST /host/widgets/{instance_id}/actions/{action}
```

Do not keep the final official Kanban limited to direct consumption of:

```
/host/integrations/servers/{server_id}/views/{view_id}/stream
```

The instance-aware runtime should:

- resolve the widget instance
- read `serverId`, `viewId`, `widgetState`, and stream state
- subscribe internally to backend data
- validate Record compatibility
- render Maud fragments
- emit Datastar fragment and signal patches

2.8.5. Data contract

2.8.5.1. Required columns

The Kanban only accepts views centered on `Record` shape with:

- `id`
- `quantity`
- `head`
- `body`

If those are not present, the widget must reject the input with a short mismatch state.

2.8.5.2. Optional columns

If present, these should be consumed:

- `bucket`
- `bucket_label`
- `start_date`
- `end_date`
- `estimate_consumption`
- `actual_consumption`
- `estimate_hours`
- `actual_hours`
- `assignee_id`
- `assignee_name`
- `parent_id`
- `depth`
- `children_count`
- `comments_count`
- `comments_preview`
- `last_comment_preview`

If absent:

- the board still functions with core Record behavior
- dependent UI should hide cleanly
- the widget must not invent values

2.8.5.3. Current schema truth

Current stable Record truth is still:

- `id`
- `quantity`
- `head`
- `body`

That means:

- full CRUD for `head`, `body`, and `quantity` is already conceptually valid
- richer fields require view projection and recommended sidecar tables

2.8.6. Quantity mapping

Current Kanban column mapping must stay compatible:

- `0` => Backlog
- `-1` => Next
- `-2` => WIP
- `-3` => Review
- `1` => Done

This mapping is odd, but should remain for compatibility unless a broader migration is chosen.

2.8.7. Shell structure

The Kanban shell should be stable and Maud-owned.

Stable regions:

- widget header
- connection and action toolbar
- board viewport
- optional footer or status region

Preferred fragment patch targets:

- `#kanban-header-meta`
- `#kanban-toolbar-state`
- `#kanban-columns`
- `#kanban-empty-or-error`
- `#kanban-create-sheet`
- `#kanban-edit-sheet`

Preferred signal-owned state:

- connection badge state
- paused or running state
- global body display mode
- per-card body display mode
- collapsed columns
- column widths
- active sheet or drawer
- local filters
- local drafts that are safe to persist

2.8.8. Technology ownership

2.8.8.1. Maud

Maud should own:

- shell
- loading state
- locked state
- mismatch state
- compact error state
- header and board structure
- columns and cards
- create and edit surfaces
- empty states

2.8.8.2. Datastar

Datastar should own:

- local UI signals
- connection/auth/runtime indicators
- collapse state
- body mode state
- small stable-shell reactivity
- signal-driven persistence glue into `widgetState`

2.8.8.3. JavaScript

JavaScript should own:

- drag and drop between columns
- pointer-driven column resize
- focus-sensitive editing
- imperative scroll behavior
- optimistic move choreography where needed

2.8.9. Persistence

Persist runtime ergonomics in `widgetState` through the host:

- column widths
- collapsed columns
- global body display mode
- per-card body display mode
- local filters
- stream paused preference

Do not persist business truth in `widgetState`.

Do not persist widget ergonomics in sidecar business tables.

2.8.10. Card content model

Core card content:

- `head` as title
- `body` as description
- `quantity` as Kanban status

Optional content extensions:

- bucket
- dates
- estimate
- actual work
- assignee
- hierarchy
- comments

Body display modes per card:

- hidden
- excerpt
- full

There should also be a global control to switch body display mode across the board.

2.8.11. CRUD model

2.8.11.1. Create

Create flow should support:

- creating the core `record`
- optionally writing `record_extension`
- optionally writing `record_link`

Prefer a semantic action endpoint for this instead of forcing multi-step client orchestration.

2.8.11.2. Edit

Edit flow should support:

- editing `head`
- editing `body`
- editing supported optional metadata when present

2.8.11.3. Move

Move flow should:

- patch `record.quantity`
- update UI optimistically if useful
- reconcile against server truth

2.8.11.4. Delete

Delete flow should delete the core `record`.

If sidecar tables are implemented with cascading deletes, that should remove:

- `record_extension`
- `record_link`
- `record_comment`
- `record_worklog`

2.8.12. Optional feature mapping

Use the recommended sidecar strategy from `web_components.typ`.

Mapping:

- bucket => `record_extension` namespace `task.bucket`
- start/end date => `record_extension` namespace `task.schedule`
- estimate => `record_extension` namespace `task.effort`
- simple assignee metadata => `record_extension` namespace `task.assignment`
- hierarchy => `record_link`
- comments => `record_comment`
- accumulated real work => `record_worklog`

2.8.13. Connection and auth behavior

The board must clearly communicate:

- whether SSE is connected
- whether stream delivery is paused
- whether auth has fallen
- whether host configuration is missing

Rules:

- do not implement widget-local login UI for the standard host-mediated flow
- use the general web login
- if auth is missing, show locked or waiting state
- if host config is missing, show misconfigured state
- if `meta.streams.enabled` is `false`, stop reconnecting until re-enabled

Required controls:

- pause receiving SSE data
- toggle connection or reception behavior if the runtime supports it

2.8.14. Layout and scrolling

The board should:

- occupy the full height of the component
- allow horizontal scrolling when columns overflow
- allow internal component scrolling like a page
- allow individual column scrolling

Column behavior should support:

- per-column scroll
- resize by dragging
- collapse or minimize

2.8.15. UX states

Required explicit states:

- loading
- empty
- mismatch
- locked/auth missing
- misconfigured
- live
- paused
- reconnecting or disconnected

2.8.16. Patch strategy

HTML fragment patches are preferred for:

- columns
- card lists
- empty/error panels
- create/edit surfaces
- structural board updates

Signal patches are preferred for:

- connection state
- pause state
- body mode state
- collapse state

- width metadata
- header/toolbar status

Rules:

- do not patch the entire document
- do not stream script tags as normal update fragments
- patch stable inner targets
- keep JS loaded once, then patch HTML and signals around it

2.8.17. Functional checklist

Must support:

- host `widgetState` persistence
- SSE view data consumption
- Record-shape validation
- CRUD on Record
- quantity-based column mapping
- full-height component layout
- horizontal board scroll
- internal component scroll
- per-column scroll
- per-column resize
- column collapse
- per-card body mode
- global body mode
- pause/reconnect/live state visibility
- backend/auth failure visibility
- delete cards

Schema-dependent advanced features:

- comments
- hierarchy
- assignee metadata
- bucket metadata
- dates
- estimate and actual work

2.8.18. Recommended implementation phases

Phase 1:

- strict Record validation
- stable Maud shell
- official instance-aware stream
- HTML fragments for columns and cards

- Datastar signals for body mode, collapse, and status

Phase 2:

- drag/drop
- resize
- full CRUD
- persistence polish
- connection/auth UX polish

Phase 3:

- comments
- hierarchy
- bucket/date/effort/assignment writes
- worklog support

2.8.19. Final position

The official Record Kanban should be built as a server-shaped board with a rich local UI layer.

That means:

- `Engineer` for structure and data authority
- `Clown` traits for local board ergonomics
- sidecar tables for richer task metadata
- host-owned auth and runtime state
- semantic actions where multi-table workflows become complex

2.9. Transfer Proposals

transfer	DATA TYPE
id	INTEGER
records_received	JSON
records_contributed	JSON
agreement	JSON
agreement_time	TIMESTAMP
transfer_confirmation	JSON
transfer_time	TIMESTAMP

Transfer Proposals are a way of exchanging record quantities between different DNAs. It is thought to be used for executing economic exchanges like buying goods and services, donations, organizing events, work tasks…

‘records_received’ is a collection of records and their quantities that will interact with our records, things you will receive. ‘records_contributed’ are the records you will contribute

and their quantities, to the records of other parties, can be more than one party. So you receive 1 in your 'Apple' Record's quantity, and give out 5 moneys to the other party involved.

'agreement' is a collection of agreement by all parties involved. 'agreement_time' is the moment every party agreed for the conditions of the trade, who will receive what. 'transfer_confirmation' is also a collection but with a confirmation from all parties that the transfer was successful, and 'transfer_time' for saving the event's moment.

2.10. History

> This is from the old Postgres Schema and idea of History, it is kept here whilst it is in TODO.

history	DATA TYPE
id	INTEGER
record_id	INTEGER
change_time	STRING
old_quantity	FLOAT
new_quantity	FLOAT

History is a table that automatically logs the change of a record's quantity, to use the 'sum' table. It is possible to do a history of Karma too, that would be useful probably.

2.11. Dna

In practice, each DNA is a different database. In essence, a different Lince behavior. Be it by swapping databases the app is pointing to or aggregating data from multiple databases. It is the data the program has access to understand how to function, what karma rules are applied to what records.

Cough

3. Sickness

'Morbus est medicus naturae': Disease is the doctor of nature, illness forces correction

— ChatGPT (circa 2025)

Lince is not great, it always needs to get better, until it dies, then it doesn't. The way to do it is by giving it medicine (teaching people how to use it) while we cure it (making it simpler and featureful).

For detailed tasks, go to the end of this book.

3.1. Good practices

Making a Lince DNA, modeling items and actions to perform is a continuous effort, if it makes sense to you. With that said, here are some ideas to get started:

– Create a view or more with all the data and configuration for managing your DNA, so Configurations, Collections, Views…

Karma

Until Lince has Deterministic Simulation Testing (DST), which in this context is a way to grab a DNA and simulate years of usage, you have to be mindfull of the Command table you produce, every command you set may possibly break your system if you don't tidy things up. If you have logic of running a command every hour if one record has quantity > X and you forget about it, any simple change will trigger it, so put guardrails for running things, be it Commands, Queries or even changing Record Quantities. Changes might cascade and deliver unforeseen consequences.

With DST this is easier to do, the plan is to have a containerized environment that runs a simulation of your system, isolated from the outside world. Your DNA (your personal configuration of Lince) is a seed that can be run multiple times arriving at the same result (hopefully). Being able to run it with a high speed, changing the date (affecting the Frequency table) and record quantities will bring reproducible results. When you want to add something to your DNA you can check it's effects with a simulation and get info if it breaks anything in an edge case.

Command

Lince works with Sqlite files for it's database. It is recomended to frequently backup your DNAs, weekly, daily or hourly if you are paranoid. If some error or mistake happens, your information is safe.

Personal Tasks – Create Views for understanding your tasks, select the ones that have negative quantity, check this SQL:

```
SELECT * FROM record WHERE quantity < 0 AND LOWER(head) LIKE '%task%'
```

– One can use the 'head' column in 'record' table as a tag holder. So Record's heads with Items and Tasks that have negative quantities appear in my 'Negatives' view.

Finance Currently not very featureful. You can model in Records your monthly earnings and expenditures and make a Karma with a condition of (Records of Earnings) - (Records of Expenditures)

Live

4. The Death of Lince

Death is only the beginning

— The Mummy (1999)

Most software is repetitive Create, Read, Update, Delete (CRUD) operations. They exist as islands on the big internet sea. Lince tries to build a Pangea.

We can create a good general base for basic CRUD software, and continue to expand beyond that. The diversity in what can be created with it is the real power.

People will have access to a free and open source todo app, event planner, financial analysis platform, industrial production planning and more; all with a change of data (DNA) and maybe some extensions.

Once the operational part of life is modeled, automated and interconnected, the effects of having built a robust and generic base for interaction with systems will start to show. If there is no incentive to rebuild a management system/platform (because there is a clearly G.O.A.T.ed one), more efforts will have to be focused on new highly specific innovative and optmizing ideas. People will enjoy having previously centralized, private and paywalled capabilities now in close reach for free.

Think of Amazon and Uber, they are companies that get a cut, to orchestrate something that computationally a global peer-to-peer network personal devices could easily manage. Think of how much your much time your devices spend processing your online purchases and how much it could be plugged into the LNHM (Lince-Net-Hive-Mind).

Amazon is a platform for Records of item contributions, your purchases are Transfer Proposals. The rides you make on Uber are Transfer Proposals of money for transportation from point A to B. Any automation they make inside their platforms is limited to the data they have on you, which is a miniscule amount of your whole life (at least I hope it is). If you where to set up hacky ways to use those platforms automatically to fit your needs, you would be doing it to every app you have, connecting them in ways they where not meant to, possibly programming something to organize all those changing external apps. It doesn't need to be that inefficient, you can have one central program that you can trust to work offline for you, acting just the way you want, featureful, built by many people, for it is the main one. The model of Lince can be a superset of many apps, and can spark many different markets and support people with it's usage.

Lince is actually not the app you install, that's a Lince Cell. When you have a group of Cells (or p2p nodes) you have a Lince Organ; people interacting to meed their needs (or others'). Lince is actually when all Cells and Organs work together to satiate everyone's Needs.

The assumption is that an app that is useful for a lot of cases and incentivizes people to help eachother, with no extra friction, is the best use of technology we can have. It will not solve anything by itself, but is is the best tool in the software part of the world.

4.1. Serious Mode

Like every good contribution, the goal is to not have to contribute anymore; because it was a one time need, or the frequent need has been solved.

Lince is a contribution to the need of having Lince, of having all production possibly connected, so there is predictability in resource usage and logistics coordination.

It's a contribution to the disorganized construction of solutions glued by the internet, centralized in platforms that worsen by the day.

It's a contribution to understanding the whole of possibilities of how you can connect with other people, after seeing needs you didnt know others had, sparking ideas of contributions you didn't know you could make.

The death of Lince is the death of the need of Lince. If there is ever any configuration of resources that makes Lince useless, then it's job is fullfilled, there is nothing else to be done, no commit, no push, just vibes. Untill then, a system for self organization that can turn into a dance of the world is, to some, something useful and exciting to build.

A team is being assembled, to fight the battle of killing Lince. Will you join the cult?